

Verteiler-Platine: Pi, HiFiBerry, RC522, Taster, Encoder, Display

Eine Lochraster-HAT, die auf den 40-Pin-Header des Raspberry Pi 3B+ gesteckt wird, den HiFiBerry Amp unverändert durchreicht und alle übrigen Komponenten sauber verkabelt.

Bauteile

Alles Standardware, keine Sonderanfertigung nötig.

1×

Proto-HAT-Lochrasterplatine

HAT-Formfaktor mit vorgebohrtem 40-Pin-Footprint + Befestigungslöchern, freies Rasterfeld

1×

2×20 Stacking-Header (extra hoch)

Reicht alle 40 Pins nach oben durch, damit der HiFiBerry weiter aufgesteckt werden kann

1×

2×20 Buchsenleiste

Unterseite der HAT, steckt auf den Pi

5×

Stiftleisten/JST-Stecker

Für RC522 (7-pol.), Taster (2× 2-pol.), Encoder (5-pol.), Display (8-pol.)

—

Litzendraht + Lötzinn

Punkt-zu-Punkt-Verdrahtung auf dem freien Rasterfeld, keine Leiterbahnplanung nötig

Aufbau als Stapel

Der HiFiBerry braucht ebenfalls den vollen Header - deshalb reicht unsere Platine alle 40 Pins unverändert nach oben durch und zapft elektrisch nur das an, was sie wirklich braucht.

HiFiBerry Amp

steckt oben auf den Stacking-Header, unverändert

⋮ 40-Pin Durchreichung

Verteiler-Platine (diese HAT)

zapft: SPI-Bus + CE1 (RC522), CE0/DC/RST (Display), GPIO5/6 (Taster), GPIO17/22/27 (Encoder)

⋮

Raspberry Pi 3B+

40-Pin-GPIO-Header

Von der Verteiler-Platine gehen 5 Steckverbinder seitlich ab → siehe unten

40-Pin-Header, komplett

Physische Pin-Nummern (nicht BCM) - so wie sie beim Auflöten der Buchsenleiste tatsächlich vor einem liegen. Ungefärbte/blasse Pins bleiben frei.

1 3V3 Stromversorgung	2 5V nicht verwendet (Module sind 3.3V)
3 GPIO2 I2C SDA · HiFiBerry	4 5V nicht verwendet
5 GPIO3 I2C SCL · HiFiBerry	6 GND
7 GPIO4 frei	8 GPIO14 frei (TXD)
9 GND	10 GPIO15 frei (RXD)
11 GPIO17 Encoder CLK	12 GPIO18 I2S BCLK · HiFiBerry
13 GPIO27 Encoder DT	14 GND
15 GPIO22 Encoder SW	16 GPIO23 frei
17 3V3 Stromversorgung	18 GPIO24 Display DC
19 GPIO10 SPI MOSI (Bus)	20 GND
21 GPIO9 SPI MISO (Bus)	22 GPIO25 Display RST
23 GPIO11 SPI SCLK (Bus)	24 GPIO8 Display CS (CE0)
25 GND	26 GPIO7 RC522 CS (CE1)
27 ID_SD reserviert (HAT-EEPROM)	28 ID_SC reserviert (HAT-EEPROM)
29 GPIO5 Taster Weiter	30 GND
31 GPIO6 Taster Zurück	32 GPIO12 frei
33 GPIO13 frei	34 GND
35 GPIO19	36 GPIO16

I2S LRCLK · HiFiBerry

frei

37 GPIO26
RC522 RST

38 GPIO20
I2S DIN · HiFiBerry

39 GND

40 GPIO21
I2S DOUT · HiFiBerry

● von dieser HAT genutzt ● HiFiBerry (nur Durchreichung) ● Stromversorgung ● GND

● frei / reserviert

GPIO18/19/20/21 sowie GPIO2/3 gehören dem HiFiBerry (I2S + I2C) - die Platine legt darauf bewusst nichts, auch nicht die Display-Hintergrundbeleuchtung (die geht direkt an einen 3.3V-Pin, nicht an GPIO18).

Steckverbinder im Detail

Was an welchen physischen Header-Pin geht, pro Modul. „Bus“ markiert die drei SPI-Leitungen (SCK/MOSI/MISO), die sich RC522 und Display teilen - getrennt bleiben nur die Chipselect-Leitungen (CE1 bzw. CE0).

RC522 (RFID, SPI CE1, 7-polig)

MODUL	HAT-PIN · BCM	
3V3	Pin 1/17	nicht 5V!
GND	beliebig	z.B. Pin 6
RST	Pin 37 · GPIO26	
SDA/C S	Pin 26 · GPIO7	
SCK	Pin 23 · GPIO11	Bus
MOSI	Pin 19 · GPIO10	Bus
MISO	Pin 21 · GPIO9	Bus

Taster „Weiter“ (2-polig)

MODUL	HAT-PIN · BCM	
A	Pin 29 · GPIO5	
B	Pin 30 · GND	

Taster „Zurück“ (2-polig)

MODUL	HAT-PIN · BCM	
A	Pin 31 · GPIO6	
B	Pin 30 · GND	

Dreh-Encoder KY-040 (5-polig)

MODUL	HAT-PIN · BCM	
CLK	Pin 11 · GPIO17	
DT	Pin 13 · GPIO27	
SW	Pin 15 · GPIO22	
+	Pin 1/17 · 3V3	
GND	Pin 14 · GND	

3.5" Display (SPI CE0, 8-polig, kein Touch)

MODUL	HAT-PIN · BCM	
VCC	Pin 17 · 3V3	
GND	Pin 20 · GND	
SCK	Pin 23 · GPIO11	Bus
MOSI	Pin 19 · GPIO10	Bus
MISO	Pin 21 · GPIO9	Bus
CS	Pin 24 · GPIO8	
DC	Pin 18 · GPIO24	
RST	Pin 22 · GPIO25	

Aufbau-Reihenfolge

- 1 2×20-Buchsenleiste auf die Unterseite der Proto-HAT löten (steckt später auf den Pi), den 2×20-Stacking-Header direkt darüber in dieselben Löcher auf die Oberseite.
- 2 Fünf Stiftleisten/JST-Buchsen an den Rand der Platine löten: RC522 (7-pol.), je einen 2-poligen für die beiden Taster, Encoder (5-pol.), Display (8-pol.).
- 3 Jeden Steckverbinder-Pin per Litzendraht punktweise mit dem passenden Header-Pin verbinden (Tabellen oben) - Drahtführung ist unkritisch, es gibt keine Hochfrequenzsignale außer dem SPI-Bus, der ohnehin kurz bleibt.
- 4 SPI-Bus-Pins (SCK/MOSI/MISO, physische Pins 23/19/21) sind Sternpunkte: RC522 und Display hängen beide daran, aber mit getrennten CE-Leitungen (CE1 bzw. CE0).
- 5 Vor dem ersten Einstecken: Durchgangsprüfung mit Multimeter, dass keine zwei Pins der Buchsenleiste versehentlich kurzgeschlossen sind (typischer Lochraster-Fehler).
- 6 HAT auf den Pi stecken, HiFiBerry oben auf den Stacking-Header, dann RC522/Taster/ Encoder/Display an ihre Steckverbinder - Display-Verkabelung selbst siehe docs/hardware.md im Repo (Touch-Leitungen bewusst nicht verbunden).

Ergänzt docs/hardware.md im OwlBox-Repo - Pin-Zuordnung ist mit der dortigen Konfiguration (config/config.example.yaml) identisch.

Hardware-Aufbau: Komponenten, Pinouts & Treiber-Setup

Vollständige Referenz zu jeder Komponente einzeln - Display-Identifikation, GPIO-Belegung, HiFiBerry/RC522/Taster/Encoder-Verkabelung und die Treiber-Installation fürs SPI-Display. Ergänzt die Verteiler-Platine oben.

Zielhardware:

- Raspberry Pi 3B+
- HiFiBerry Amp (I2S-Verstärker-HAT)
- RC522 RFID-Modul (SPI, 13.56 MHz)
- 3.5" SPI-Touchscreen, 480×320, mit Stylus, 26-Pin-Header - **sehr wahrscheinlich ein "MHS-35"/"tft35a"-Klon** (ILI9486 + XPT2046, wird unter vielen Markennamen identisch verkauft). **Touch bleibt deaktiviert**, siehe unten.
- 2 Taster (vor/zurück)
- 1 Dreh-Encoder mit Druckschalter (Lautstärke / Pause)

Alle Pin-Angaben sind BCM-Nummerierung und entsprechen den Defaults in `config/config.example.yaml`. Wer andere Pins verdrahtet, passt einfach die `gpio:` / `rfid:` Sektion in `config/config.yaml` an.

Verteiler-Platine: `docs/hat-wiring.html` (im Browser öffnen) zeigt den kompletten Schaltplan als Lochraster-HAT - inkl. 40-Pin-Belegung, Stapelaufbau mit HiFiBerry-Durchreichung und Steckverbinder-Pinouts pro Modul. Die Pin-Zuordnung dort ist identisch mit den Tabellen unten.

Woran die Identifikation hängt

480×320px, Stylus im Lieferumfang, 26-Pin-Anschluss und der (leicht falsch übersetzte) Hinweis "sonst wird die Touch-Funktion ausgebrannt" sind das exakte Wortmuster, das auf praktisch jedem Amazon-Listing für diese Referenzplatine steht - verkauft unter vielen Namen (Kuman, OSOYOO, Elegoo, SunFounder, "MPI3508", generische "3.5 Zoll Display"-Listings), aber elektrisch identisch: **ILI9486-Controller fürs Display, XPT2046 für Touch**, Treiber über das `goodtft/LCD-show` - Installationsskript (`MHS35-show` bzw. `tft35a` -Overlay). Falls der beiliegende

Download-Link der Anleitung einen anderen Treibernamen nennt, bitte kurz Bescheid geben - dann passe ich das unten an.

Wichtiger Hinweis: physische Steckplatz-Kollision mit dem HiFiBerry

Das Display wird laut Beschreibung direkt auf den 26/40-Pin-GPIO-Header gesteckt ("mit einem 26-poligen SPI-Anschluss ... bitte über den ersten Anschluss anschließen"). Der HiFiBerry Amp braucht aber **ebenfalls** einen direkten Sitz auf demselben Header (I2S ist empfindlich gegenüber langen/zusätzlichen Steckverbindern). **Beide gleichzeitig aufstecken geht nicht**, sofern keins der beiden Boards einen sauberen Pass-Through- Header mitbringt (bei diesem günstigen Display-Typ i.d.R. nicht der Fall).

Lösung: Das Display nicht aufstecken, sondern per Jumper-/Dupont-Kabeln verdrahten. Der 26-Pin-Header auf der Display-Platine ist ein normaler 2.54mm-Pfostenverbinder - er lässt sich genauso gut mit einzelnen Kabeln verbinden wie durch direktes Aufstecken. Das hat zwei Vorteile:

1. Der HiFiBerry sitzt normal direkt auf dem Pi, das Display hängt per Kabel daneben.
2. **Es müssen nur die tatsächlich gebrauchten Leitungen verbunden werden** - die Touch-Leitungen (CE1/PENIRQ) werden dabei einfach **gar nicht erst angeschlossen**. Das deaktiviert Touch zusätzlich auf Hardware-Ebene, nicht nur per Software/Overlay.

Nur diese Leitungen vom Display-Header zum Pi verbinden:

DISPLAY-PIN (26-PIN-HEADER)	PI-PIN (BCM)	ZWECK
VCC	3.3V	Versorgung
GND	GND	Masse
SCK	GPIO11	SPI-Takt (mit RC522 geteilt)
MOSI (SDI)	GPIO10	SPI (mit RC522 geteilt)
MISO (SD0)	GPIO9	SPI (mit RC522 geteilt)
CS/CE0	GPIO8	Display-Chipselect
DC/RS	GPIO24	Data/Command (Standardwert des tft35a-Overlays)
RST	GPIO25	Reset (Standardwert des tft35a-Overlays)

DISPLAY-PIN (26-PIN-HEADER)	PI-PIN (BCM)	ZWECK
LED/Backlight	3.3V direkt , nicht an einen GPIO	siehe unten
T_CLK, T_CS, T_DIN, T_D0, T_IRQ (Touch)	nicht anschließen	Touch bleibt so auch elektrisch inaktiv

Zur Hintergrundbeleuchtung: Je nach Fertigungscharge ist die LED-Leitung bei diesem Board-Typ entweder fest verdrahtet oder für Software-Dimmen auf einen GPIO gelegt (öfter berichtet: GPIO18 - genau der Pin, den der HiFiBerry für die I2S-Bit-Clock braucht). Da wir per Kabel verdrahten, einfach **die LED-Leitung direkt an einen 3.3V-Pin des Pi anschließen** statt an einen GPIO - Beleuchtung ist dann dauerhaft an (Dimmen brauchen wir für eine reine Infoanzeige ohnehin nicht) und GPIO18 bleibt frei für den HiFiBerry.

GPIO-Belegung im Überblick

FUNKTION	BCM PIN	GENUTZT VON
I2S BCLK	18	HiFiBerry (Display-Backlight bewusst NICHT hierauf gelegt, s.o.)
I2S LRCLK	19	HiFiBerry
I2S DIN	20	HiFiBerry
I2S DOUT	21	HiFiBerry
I2C SDA	2	HiFiBerry (Amp-Steuerung)
I2C SCL	3	HiFiBerry (Amp-Steuerung)
SPI0 SCLK/MOSI/MISO	11/10/9	Display + RC522 (gemeinsamer Bus)
SPI0 CE0	8	Display (TFT-Chipselect)
SPI0 CE1	7	RC522 (<code>rfid.spi_device: 1</code>) - frei, da Touch nicht verdrahtet
Display DC	24	Display
Display RST	25	Display
RC522 RST	26	RC522 (<code>rfid.reset_pin</code>)
Taster Weiter	5	Taster
Taster Zurück	6	Taster
Encoder CLK	17	Encoder
Encoder DT	27	Encoder

FUNKTION	BCM PIN	GENUTZT VON
Encoder SW	22	Encoder

HiFiBerry Amp

Der HiFiBerry belegt die I2S-Pins (BCM 18/19/20/21) sowie ggf. I2C (BCM 2/3) zur Verstärkersteuerung.

In `/boot/firmware/config.txt` (bzw. `/boot/config.txt` auf älteren Images):

```
dtparam=audio=off
dtoverlay=hifiberry-amp
```

(Für andere HiFiBerry-Varianten den passenden Overlay-Namen verwenden, z.B. `hifiberry-dacplus` für ein reines DAC+. Nach der Änderung neu starten.)

Danach mit `aplay -L` und `amixer -c 0 scontrols` das ALSA-Device bzw. den Mixer-Namen prüfen und in `config.yaml` unter `audio.alsa_device` / `audio.mixer_control` eintragen (Amp/Amp2 nutzen meist `Digital`, manche Boards `PCM` oder `Master`).

RC522 RFID-Leser (SPI, CE1)

RC522 PIN	RASPBERRY PI
VCC	3.3V (nicht 5V!)
GND	GND
RST	GPIO26 (<code>rfid.reset_pin</code>)
SDA (CS)	GPIO7 / CE1
SCK	GPIO11 (mit Display geteilt)
MOSI	GPIO10 (mit Display geteilt)
MISO	GPIO9 (mit Display geteilt)
IRQ	nicht verbunden

Der RC522 liegt hier bewusst auf **CE1 (GPIO7)** statt CE0, weil das Display CE0 belegt. In `config.yaml`: `rfid.spi_device: 1`.

SPI muss aktiviert sein (macht `scripts/install.sh` bereits via `raspi-config nonint do_spi 0`, alternativ `sudo raspi-config` → Interface Options → SPI).

Taster (vor/zurück)

Als Taster kommen Cherry-MX-Switches (3-Pin-Variante) zum Einsatz. Elektrisch sind das ganz normale Momentary-Schalter (schließt nur beim Drücken, öffnet sonst) - dieselbe Verdrahtung wie jeder andere Taster:

- Von den 3 Pins sind nur die **beiden Metall-Pins** die elektrischen Kontakte (die sich diagonal gegenüberliegen); der dritte, meist aus Kunststoff, ist nur ein mechanischer Halteclip fürs Gehäuse/die Platine und hat keine elektrische Funktion - er muss nirgends angeschlossen werden.
- Jeweils einer der beiden Metall-Pins an GPIO, der andere an GND. Kein externer Widerstand nötig, der interne Pull-up wird von gpiozero aktiviert.
- Da Cherry-MX-Switches (anders als billige Blechtaster) sehr sauber prellen, reicht die Default-Entprellzeit (`gpio.bounce_time`, 50ms) komfortabel aus.

FUNKTION	BCM PIN
Zurück	6
Weiter	5

Kurz drücken springt zum vorherigen/nächsten Track. Gedrückt halten (länger als `gpio.seek_hold_seconds`, Default 0.4s) spult stattdessen im aktuellen Track vor/zurück, in Schritten von `gpio.seek_step_seconds` (Default 10s) - kein Trackwechsel, solange gehalten wird.

Dreh-Encoder mit Taster (KY-040)

ENCODER PIN	RASPBERRY PI
CLK	GPIO17
DT	GPIO27
SW	GPIO22
+	3.3V
GND	GND

Das KY-040-Modul bringt eigene Pull-up-Widerstände für CLK/DT mit; die zusätzlich von gpiozero aktivierten Pull-ups des Pi stören dabei nicht (einfach parallel). Für den Taster (SW) hat das Modul in der Regel **keinen** eigenen Pull-up - das übernimmt `gpiozero.Button(pull_up=True)` in `owlbox/controls/gpio_controls.py`, hier also nichts weiter nötig.

Drehen ändert die Lautstärke (Schritweite `audio.volume_step`), Drücken schaltet Play/Pause um. Ein langer Druck (`gpio.shutdown_hold_seconds`, Default 4s) fährt den Pi sicher herunter - praktisch für ein Kindergerät ohne Zugriff auf ein Terminal. Auf 0 setzen, um das abzuschalten.

Damit Shutdown/Neustart (auch über die Web-UI unter Einstellungen bzw. über einen "Pi neu starten"/"WLAN aus"-Funktions-Chip, siehe unten) ohne Passwortabfrage funktionieren, braucht der Service-User `owlbox` passwortloses sudo dafür, z.B. in `/etc/sudoers.d/owlbox`:

```
owlbox ALL=(ALL) NOPASSWD: /sbin/shutdown, /usr/bin/nmcli
```

3.5" SPI-Display: Treiber (ohne Touch)

Diese Board-Familie (tft35a/MHS-35) funktioniert **nicht** über einen direkten KMS-Grafikausgang, sondern über einen Trick: der Pi bekommt per `config.txt` einen "unsichtbaren" virtuellen HDMI-Ausgang in der Auflösung des Displays vorgegaukelt, X11/Chromium rendern ganz normal dorthin, und ein kleines Hilfsprogramm (`fbcp`) kopiert das Bild laufend per SPI aufs eigentliche TFT. Für den Rest des Systems (inkl. unserem `scripts/kiosk.sh`) sieht das aus wie ein ganz normaler Bildschirm.

Empfohlener Weg: statt `config.txt` von Hand zu editieren, den Treiber-Installer benutzen, den der Verkäufer laut Artikelbeschreibung verlinkt ("kostenlose Treiberinstallation und Tutorials sind verfügbar"), oder alternativ das quelloffene `goodtft/LCD-show`-Skript (auf GitHub, Skriptname i.d.R. `MHS35-show` oder `LCD35-show`) - der Installer setzt automatisch die passenden `config.txt`-Werte für genau dieses Board, kompiliert/installiert `fbcp` und richtet den Autostart ein. Passenden Namen/Link ggf. auf dem beiliegenden Handzettel oder in der Amazon-Produktbeschreibung ("siehe Bild unten für Details") nachsehen.

Danach nur einen Schritt selbst nachziehen: in `/boot/firmware/config.txt` die vom Installer eingetragene Touch-Zeile wieder entfernen/auskommentieren - sie beginnt mit `dtoverlay=ads7846,...`. Ohne diese Zeile lädt der Kernel den XPT2046-Touch-Treiber gar nicht erst, Touch ist damit auch softwareseitig aus

(zusätzlich zur ohnehin nicht verdrahteten Touch-Leitung von oben). Die restlichen vom Installer gesetzten Zeilen (virtueller HDMI- Modus, `dtoverlay=tft35a:rotate=...` o.ä., SPI aktivieren) unverändert lassen.

Zur Referenz, wie diese Zeilen ungefähr aussehen (der Installer setzt sie automatisch, exakte Werte können je nach Skriptversion leicht abweichen):

```
hdmi_force_hotplug=1
hdmi_group=2
hdmi_mode=87
hdmi_cvt=480 320 60 6 0 0 0
hdmi_drive=2
dtparam=spi=on
dtoverlay=tft35a:rotate=90
```

Und als systemd-Service für `fbcp`, falls der Installer keinen eigenen Autostart einrichtet (Vorlage liegt in `systemd/owlbox-fbcp.service`, Pfad ggf. anpassen falls der Installer `fbcp` woanders ablegt):

```
sudo cp systemd/owlbox-fbcp.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now owlbox-fbcp.service
```

Wichtig: legacy Grafiktreiber statt Wayland/labwc

`fbcp` liest von `/dev/fb0` - das gibt es unter dem modernen KMS-Grafiktreiber (`vc4-kms-v3d`, Standard seit Raspberry Pi OS Bullseye/ Bookworm mit Wayland/labwc) meist nicht mehr in nutzbarer Form. Für dieses Display-Familie also in `/boot/firmware/config.txt` den KMS-Treiber deaktivieren bzw. auskommentieren:

```
#dtoverlay=vc4-kms-v3d
```

und über `sudo raspi-config` → System Options → Boot / Desktop den Pi auf den klassischen X11-Desktop (nicht Wayland) stellen. Das ist auch der Weg, den praktisch alle Tutorials für dieses Board beschreiben - Wayland/labwc lohnt sich hier nicht zu erzwingen. (Alternative für alle, die KMS/Wayland behalten wollen: der Fork `fbcp-ili9341`, der über DRM statt `/dev/fb0` liest - aufwändiger einzurichten, hier nicht weiter dokumentiert.)

Falls es doch ein anderes Board ist

Sollte die Anleitung/Download-Karte, die dem Display beilag, einen anderen Overlay-/Treibernamen nennen als oben: gerne den genauen Namen schicken, dann passe ich `config.txt` und die Pin-Tabelle entsprechend an.

Kiosk-Autostart (Chromium fullscreen)

`scripts/kiosk.sh` startet Chromium im Kiosk-Modus gegen `http://localhost:5000/` - das funktioniert unverändert, sobald `fbcp` läuft und X11 (nicht Wayland) aktiv ist, weil Chromium dann ganz normal auf den virtuellen HDMI-Ausgang rendert.

Autostart einrichten:

```
mkdir -p ~/.config/systemd/user
cp /opt/owlbox/systemd/owlbox-kiosk.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now owlbox-kiosk.service
sudo loginctl enable-linger $USER # optional: startet auch ohne aktive Anmeldung
```

Alternativ über die Autostart-Datei der Desktop-Umgebung:

`~/.config/lxsession/LXDE-pi/autostart` um die Zeile `@/opt/owlbox/scripts/kiosk.sh` ergänzen.